

CSE312: Database Management Systems Lab

Lab 02: PostgreSQL DDL Statements

1. Objective

The objectives of this experiment are:

- To understand the purpose of Data Definition Language (DDL) in PostgreSQL.
- To create and manage databases, schemas, and tables using PostgreSQL commands.
- To define table columns with suitable data types and constraints.
- To modify existing database objects using **ALTER** statements.
- To remove database objects safely using **DROP** and **TRUNCATE** statements.

2. Introduction

Data Definition Language (DDL) is a part of SQL used to define, modify, and remove database structures. In PostgreSQL, DDL statements are used to create databases, schemas, tables, constraints, indexes, views, and other database objects.

This lab focuses on the most commonly used PostgreSQL DDL statements:

- **CREATE** – creates a new database object.
- **ALTER** – modifies an existing database object.
- **DROP** – permanently removes a database object.
- **TRUNCATE** – removes all rows from a table while keeping the table structure.

Required software:

- PostgreSQL Server
- pgAdmin or psql command-line tool

Screenshot Placeholder: PostgreSQL server or pgAdmin dashboard

3. Theory

3.1 Data Definition Language

DDL: Data Definition Language is the subset of SQL used to define and manage the structure of a database. DDL commands affect database objects such as databases, schemas, tables, and constraints.

Common DDL statements are summarized below:

Statement	Purpose
CREATE	Creates a database object such as a database, schema, or table.
ALTER	Modifies the structure of an existing database object.
DROP	Deletes a database object permanently.
TRUNCATE	Removes all rows from a table without deleting the table itself.

Table 1: Common PostgreSQL DDL statements

3.2 PostgreSQL Data Types

Choosing appropriate data types is important because data types control what kind of values can be stored in each column.

Data Type	Description
INTEGER	Stores whole numbers.
SERIAL	Auto-incrementing integer, commonly used for primary keys.
VARCHAR(n)	Stores variable-length character strings with a maximum length.
TEXT	Stores variable-length text without a fixed maximum length.
NUMERIC(p, s)	Stores exact decimal values with precision and scale.
DATE	Stores calendar dates.
TIMESTAMP	Stores date and time values.
BOOLEAN	Stores true or false values.

Table 2: Frequently used PostgreSQL data types

3.3 Constraints

Constraint: A constraint is a rule applied to a table column or a group of columns to maintain data accuracy and integrity.

Common constraints include:

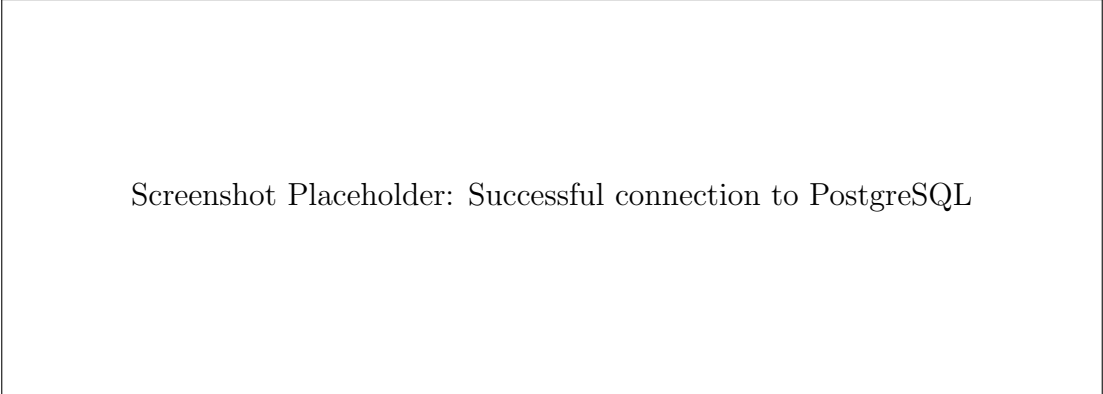
- **PRIMARY KEY:** uniquely identifies each row in a table.
- **FOREIGN KEY:** links a column to the primary key of another table.
- **NOT NULL:** prevents a column from storing null values.
- **UNIQUE:** ensures all values in a column are different.
- **CHECK:** ensures values satisfy a specified condition.
- **DEFAULT:** assigns a value automatically when no value is provided.

4. Procedure

4.1 Connecting to PostgreSQL

Open pgAdmin or the psql command-line tool and connect to the PostgreSQL server using a valid username and password.

```
psql -U postgres
```



Screenshot Placeholder: Successful connection to PostgreSQL

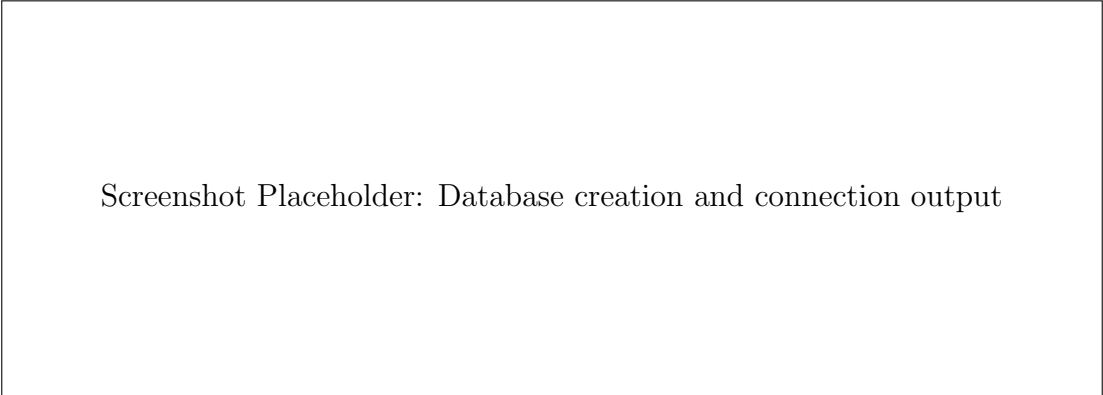
4.2 Creating a Database

Use the CREATE DATABASE statement to create a new database for this lab.

```
CREATE DATABASE university_lab;
```

After creating the database, connect to it.

```
\c university_lab
```



Screenshot Placeholder: Database creation and connection output

4.3 Creating a Schema

A schema is a logical container for database objects. It helps organize tables and other objects.

```
CREATE SCHEMA academics;
```

Set the search path so PostgreSQL uses this schema by default.

```
SET search_path TO academics;
```

4.4 Creating Tables

The CREATE TABLE statement defines a new table, its columns, data types, and constraints.

```
CREATE TABLE department (  
    dept_id SERIAL PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL UNIQUE,  
    location VARCHAR(100)  
);
```

```
CREATE TABLE student (  
    student_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    date_of_birth DATE,  
    admission_year INTEGER CHECK (admission_year >= 2000),  
    dept_id INTEGER REFERENCES department(dept_id)  
);
```

```
CREATE TABLE course (  
    course_id SERIAL PRIMARY KEY,  
    course_code VARCHAR(20) NOT NULL UNIQUE,  
    course_title VARCHAR(150) NOT NULL,  
    credit NUMERIC(3, 1) CHECK (credit > 0),  
    dept_id INTEGER NOT NULL REFERENCES department(dept_id)  
);
```

Screenshot Placeholder: Table creation statements executed successfully

4.5 Creating a Table with Composite Primary Key

A composite primary key is formed using two or more columns. It is useful for junction tables.

```
CREATE TABLE enrollment (  
    student_id INTEGER REFERENCES student(student_id),  
    course_id INTEGER REFERENCES course(course_id),  
    enrollment_date DATE DEFAULT CURRENT_DATE,  
    grade VARCHAR(2),  
    PRIMARY KEY (student_id, course_id)  
);
```

4.6 Viewing Table Structure

In psql, the `\d` command can be used to view the structure of a table.

```
\d department  
\d student  
\d course  
\d enrollment
```

Screenshot Placeholder: Table descriptions showing columns and constraints

4.7 Modifying Tables with ALTER

Use `ALTER TABLE` to modify an existing table structure.

Add a new column:

```
ALTER TABLE student
ADD COLUMN phone VARCHAR(20);
```

Rename a column:

```
ALTER TABLE student
RENAME COLUMN phone TO contact_number;
```

Change a column data type:

```
ALTER TABLE student
ALTER COLUMN contact_number TYPE VARCHAR(30);
```

Add a constraint:

```
ALTER TABLE enrollment
ADD CONSTRAINT grade_check
CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F'));
```

Screenshot Placeholder: ALTER TABLE statements and updated table structure

4.8 Renaming and Dropping Database Objects

Rename a table using `ALTER TABLE`.

```
ALTER TABLE course  
RENAME TO offered_course;
```

Drop a table only when it is no longer needed. The `IF EXISTS` clause prevents an error if the table does not exist.

```
DROP TABLE IF EXISTS temporary_table;
```

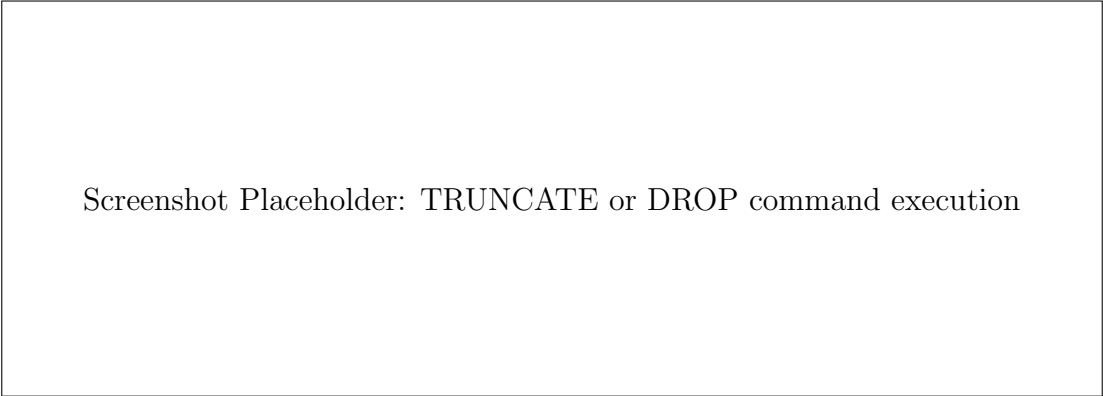
4.9 Truncating a Table

The `TRUNCATE` statement removes all rows from a table while keeping the table structure.

```
TRUNCATE TABLE enrollment;
```

If another table references the table, use `CASCADE` carefully.

```
TRUNCATE TABLE enrollment CASCADE;
```



A rectangular box with a thin black border, containing the text "Screenshot Placeholder: TRUNCATE or DROP command execution".

5. Observations

- A new PostgreSQL database was created and selected successfully.
- A schema was created to organize database objects.
- Tables were created with appropriate data types and constraints.
- Primary key and foreign key constraints were used to maintain entity integrity and referential integrity.
- Existing table structures were modified using `ALTER TABLE`.
- Rows were removed using `TRUNCATE` without deleting the table definition.

6. Lab Task

Task A: Library Management Database

Create a PostgreSQL database named `library_lab`. Then create the following tables using DDL statements:

1. `publisher`: stores publisher information.
2. `book`: stores book information and references `publisher`.
3. `member`: stores library member information.
4. `borrow`: stores borrowing records and references `book` and `member`.

Requirements:

- Use suitable data types for all columns.
- Define primary keys for all tables.
- Define foreign keys where necessary.
- Add at least one `UNIQUE`, one `NOT NULL`, one `CHECK`, and one `DEFAULT` constraint.
- Use `ALTER TABLE` to add one new column to any table.
- Capture screenshots of the SQL commands and table structures.

Screenshot Placeholder: Completed Task A DDL statements and output

Task B: Hospital Management Database

Write PostgreSQL DDL statements for a hospital management database containing the following tables: `department`, `doctor`, `patient`, and `appointment`.

Requirements:

- Each doctor must belong to a department.
- Each appointment must reference one doctor and one patient.
- Appointment dates should have a default value of the current date.
- Consultation fees must be greater than or equal to zero.
- Add suitable primary key, foreign key, unique, not null, default, and check constraints.
- Include screenshots of the commands and the final table descriptions.

Screenshot Placeholder: Completed Task B DDL statements and output

7. Conclusion

This experiment introduced PostgreSQL Data Definition Language statements for creating and managing database structures. Students practiced creating databases, schemas, and tables; defining data types and constraints; modifying table definitions; and removing or clearing database objects. These DDL skills provide the foundation for building relational database schemas before inserting and querying data in later experiments.